What Is DeFi Contract Auditor?

A plain-language guide for anyone, no technical background needed

Context Document | Draft v0.1 | October 25, 2026
Status: the product is in the design stage. This guide describes what it is intended to do.

The 30-second version

Some software holds real money and cannot be changed once it is released. These programs are called **smart contracts**. If one has a mistake, or gives someone too much power, money can vanish and there is usually no way to undo it.

DeFi Contract Auditor is a safety check you run **before** you release such a program. You give it your code. A few minutes later it tells you (1) who holds powerful 'master keys' and what the worst case would be if one were misused, (2) which parts of the code look risky and why, with proof, and (3) how to fix them, with the fixes already tested.

> **An everyday comparison**
> Think of it as a building inspector's quick walk-through before you open a vault to the public. It is useful and fast, but it does not replace the full professional inspection.

Why this exists

- **Mistakes are permanent.** Public figures from 2025 put money stolen from crypto projects at over $3 billion for the year, and the first half of 2026 was well over $1 billion again (different trackers count differently).
- **Professional reviews cost a lot.** A full smart contract security review commonly costs tens of thousands of dollars, often $25,000 to $100,000 for a typical DeFi project, and takes days to weeks plus waiting time.
- **Small teams get squeezed.** A young startup with a launch date and a small budget may skip checks, or check too late.
- **Basic scanning tools are noisy.** They produce long lists of warnings that developers learn to ignore, and they rarely explain who holds dangerous powers.

Who it is for

If you are...	You would use it to...
A founder or developer launching a DeFi or token project	Find problems cheaply before your formal audit, and explain your admin risk clearly to investors.
An independent security reviewer	Speed up your first pass over a new codebase with line-by-line leads you can verify quickly.
A compliance or risk manager (later phase)	Get a list of contract features that raise regulatory questions to take to a lawyer. This part is planned, not in the first release.

How it works, step by step

Step	What happens	In plain words
1	You upload your contract files (or a zipped project).	Like attaching a document to an email.
2	The system builds your code in a locked, disposable room with no internet.	It checks that your code actually compiles, without letting your files touch anything else.
3	It reads the code's structure and finds every 'privileged' action, such as who can create new tokens, pause everything, upgrade the code or withdraw funds.	It makes a list of everyone holding a master key and what each key opens.
4	It scores each key by how much damage it could do, and writes a worst-case sentence.	'If this one key is stolen, an attacker could do X.'
5	It runs standard security checks and looks up similar past incidents in a library.	'Has this kind of mistake caused a hack before?'
6	It drafts fixes and tests that each fix compiles and clears the problem.	Fixes are shown only if they pass their own test.
7	You get a report with exactly what was and was not checked.	No hiding the limits.

The star feature: the Privilege Map

Many large crypto losses involve someone having too much power, or one stolen key being enough to cause disaster. The **Privilege Map** is a diagram of this. On one side are the **roles** (for example, 'Owner' or 'Minter'). On the other are the **powers** (for example, 'create unlimited tokens' or 'take all funds'). Lines connect who can do what. Each role gets a risk score and a sentence anyone can read.

The code alone cannot tell the system **who actually holds** each key. A key held by one person's wallet is far riskier than one that needs three of five people to approve and waits 48 hours before acting. So you can optionally tell the tool how your keys are held, and the risk updates immediately. If you do not say, the tool assumes the worst and labels the result 'assumed'.

An example

Imagine a fictional project called **FarmVault**, where people deposit tokens to earn rewards.

- *The Privilege Map shows: Owner can withdraw all funds and create unlimited tokens, with no waiting period and no cap. Single key assumed. Risk: critical.*

- *The founder realizes the owner key is one person's wallet. She moves it to a 3-of-5 shared wallet with a 48-hour delay, and tells the tool.*
- *The score drops, because stealing the key no longer lets an attacker act instantly. The report still lists the powers, so her investors can see exactly what the team can and cannot do.*
- *Separately, the tool flags a deposit function that sends money out before it updates its records, a classic mistake that has been exploited before. It shows the matching past incident, the exact lines, and a fix that it has already tested.*

What you get

- **Privilege Map** with scores and worst-case sentences.
- **Findings** with location, severity and evidence. Every finding has at least one source to back it up: a fact found directly in your code, a cited past incident, or a test that reproduces the problem.
- **Fixes** shown as red (removed) and green (added) lines, tested before you see them.
- **Coverage statement**: what was checked, what could not be checked, and what is outside any tool's reach.
- **Exports** for developers and auditors.

What it cannot do (important)

> **Plain honesty**
> It does not guarantee your contract is safe. It is not a formal audit and gives no certificate. It is not legal advice. It cannot see off-chain things such as who really holds your keys, your team's security habits, or your marketing. It does not watch live contracts or track wallets after launch. It may miss bugs, especially unusual ones, and it may flag things that turn out to be fine.

- If the tool says 'no verified issues detected', that means only that *this analysis* did not verify any. It is never the same as 'safe'.
- The risk score is a rough indicator, not a grade. Please do not advertise it as a certification.
- Keep your formal audit. This tool is meant to make it cheaper and faster, not to replace it.

Your code and privacy

- Your code is processed in isolated, temporary environments and is kept private to your account.
- The design target is to delete uploaded code after 24 hours by default, or immediately when you press delete.
- Your code is not added to the shared library the tool uses for lookups, and it is not used to train AI models. The AI services used are meant to be under no-retention, no-training terms.
- These are design commitments for the product being specified. They must be built and independently verified before launch.

How accurate will it be?

We will measure it and publish the results. Targets for the first versions include correctly identifying at least 90% of privileged powers on a hand-labeled test set, and keeping unsupported claims (statements without a verifiable source) under 1%. Independent research shows that today's best AI agents find a meaningful share of known vulnerabilities but miss many, and results vary run to run, which is why this product relies on verification rather than on trusting an AI's say-so.

Glossary

Term	Meaning
Smart contract	A program on a blockchain that holds or moves money and runs exactly as written.
Solidity	The most common programming language for smart contracts on Ethereum and similar chains.
Audit	A professional review of code to find security problems.
Vulnerability	A mistake that an attacker can use to steal money or break the program.
Reentrancy	A classic bug where a contract can be tricked into paying out repeatedly before it updates its books.
Admin key / owner	A special account with extra powers inside a contract.
Multisig	A shared wallet that needs several people to approve an action, for example 3 of 5.
Timelock	A built-in waiting period before a powerful action can take effect, giving people time to react.
Proxy / upgradeable contract	A contract whose logic can be swapped later, which makes the key that controls the swap very powerful.
Static analysis	Examining code without running it, to spot risky patterns.
RAG (retrieval)	Looking things up in a library of past incidents before answering, so answers can point to a real source.
Invariant	A rule that must always hold, such as 'nobody can withdraw more than they deposited'.
Patch / diff	A proposed code fix, shown with removed lines in red and added lines in green.
Sandbox	A locked, disposable environment where code can be examined safely.

Common questions

Is this an audit?

No. It is an automated advisory check. A formal audit by qualified humans is still strongly recommended before launch.

Which blockchains does it cover?

The first versions focus on Solidity contracts on Ethereum-compatible (EVM) chains. Other languages and chains are later work.

How long does a check take?

The target is a few minutes for a typical small contract. Larger projects take longer.

What if my code will not compile?

The tool will tell you exactly what failed and where. It deliberately does not guess at a review of code it cannot build.

Does it check legal compliance?

Not in the first release. Regulations differ by country and change often, so this is planned for a later phase and will produce questions to take to a lawyer, not legal conclusions.

What will it cost?

Pricing has not been decided.